

Security in Cloud Networking

Task 1

In AWS, create a new security group named InstaClone-SG and configure it to allow incoming HTTP (port 80) and HTTPS (port 443) traffic only from your own IP address.

Steps followed:

1. Logged in to the AWS Management Console and opened the EC2 Dashboard, then went to "Security Groups" under Network & Security.
2. Clicked "Create security group", named it "InstaClone-SG", and added the description "Allows HTTP/HTTPS only from my IP".
3. Selected the default VPC (same VPC as the EC2 instances that will use this group).
4. Under inbound rules, added a rule with Type = HTTP, Port range = 80, and Source = "My IP", which auto-filled my current public IP address in CIDR format (e.g., 203.0.113.10/32).
5. Added a second inbound rule with Type = HTTPS, Port range = 443, and Source = "My IP" again, so only my IP could reach either port.
6. Left the outbound rules at the default (all traffic allowed), since the goal was to restrict inbound access only.
7. Reviewed the rules and clicked "Create security group". Confirmed in the "Inbound rules" tab that both HTTP and HTTPS showed a single /32 source instead of 0.0.0.0/0.

Task 2

Set up a security group for a mock Zomato-style backend server that allows incoming MySQL (port 3306) connections only from a specific IP address and blocks all other inbound traffic.

Steps followed:

1. Created a new security group named "Zomato-Backend-SG" with the description "MySQL access restricted to app server IP".
2. Since a newly created security group has no inbound rules by default, all inbound traffic was already blocked at this point — nothing needed to be removed.
3. Added a single inbound rule using the "Custom TCP" rule type, set Port range = 3306, and set Source in CIDR format to the specific allowed IP (e.g., 203.0.113.25/32) instead of 0.0.0.0/0.
4. Did not add any other inbound rules, so every other port and protocol (SSH, HTTP, HTTPS, etc.) remained blocked for this group.
5. Left the outbound rules at the default allow-all, since the backend still needs to reach the internet for OS updates and package installs.
6. Attached Zomato-Backend-SG to the mock backend EC2 instance running MySQL.
7. Verified the setup: connecting to port 3306 from the allowed IP succeeded, while a connection attempt from a different IP timed out, confirming the rule worked as intended.

Task 3

Modify an existing security group to block all outbound traffic except for DNS (port 53) and HTTPS (port 443) to simulate a strict Paytm wallet server environment.

Steps followed:

1. Opened the existing security group intended for the wallet server (renamed to "Paytm-Wallet-SG" for clarity) and went to the "Outbound rules" tab.
2. Deleted the default outbound rule that allowed all traffic (All traffic, 0.0.0.0/0), since this is what needed to be restricted.
3. Added an outbound rule with Type = DNS (UDP), Port range = 53, Destination = 0.0.0.0/0, to allow standard DNS lookups.
4. Added a second outbound rule with Custom TCP, Port range = 53, Destination = 0.0.0.0/0, to cover DNS-over-TCP for larger responses.
5. Added an outbound rule with Type = HTTPS, Port range = 443, Destination = 0.0.0.0/0, so the server could still make secure API calls.
6. Saved the rules, leaving no other outbound ports open — outbound HTTP (80), SSH, and all other traffic was now blocked by default.
7. Verified the change from the instance: a curl request to an HTTPS endpoint succeeded, while a plain HTTP request and a ping to an external host both failed/timed out, confirming outbound traffic was locked down to only DNS and HTTPS.

Task 4

Compare security groups and network ACLs by listing two key differences in how they control traffic for a Flipkart-like cloud application.

1. Level at which they operate

A security group works at the instance (network interface) level — it is attached directly to each EC2 instance, such as Flipkart's individual web or application servers, so different servers behind the same load balancer can each have their own rules. A network ACL, on the other hand, works at the subnet level — it automatically applies to every instance inside that subnet, so it acts as a broader, shared boundary for an entire tier of the application (for example, the whole private subnet holding Flipkart's backend servers).

2. Statefulness and rule direction

Security groups are stateful: if inbound traffic is allowed in (say, a customer request hitting Flipkart's web servers on port 443), the matching return traffic is automatically allowed out, so only one rule direction usually needs to be configured. Network ACLs are stateless: inbound and outbound rules must both be explicitly defined, and rules are evaluated in numbered order, so a response to an allowed inbound request will still be blocked unless a matching outbound rule exists. This makes NACLs more granular and useful as a coarse, subnet-wide safety net, while security groups remain the primary, fine-grained control on each instance.